

Diplomatura en programación web full stack con React JS

Módulo 5:

Base de datos, integración con Node.js

Unidad 1:

Introducción a las bases de datos



Presentación

En esta unidad comenzamos a trabajar con base de datos relacionales MYSQL.



Objetivos

Que los participantes logren...

- Comprender qué son las bases de datos.
- Entender y utilizar tablas, columnas y registros.
- Conocer y utilizar los tipos de datos.



Bloques temáticos

1. ¿Qué son?
2. Tablas.
3. Columnas.
4. Registros.
5. Tipos de datos.
6. Relaciones.

1. ¿Qué son?

Una base de datos **es un conjunto de datos** pertenecientes a un mismo contexto y almacenados sistemáticamente. Se pueden clasificar según la variabilidad de los datos almacenados:

- **Bases de datos estáticas:** son bases de datos de sólo lectura, utilizadas primordialmente para almacenar datos históricos
- **Bases de datos dinámicas:** son bases de datos donde la información almacenada se modifica con el tiempo, permite operaciones como actualización, adición, consulta y eliminación de datos.

Bases de datos relacionales

Éste es el modelo más utilizado en la actualidad para modelar problemas reales y administrar datos dinámicamente.

En este modelo, el lugar y la forma en que se almacenan los datos **no** tienen relevancia (a diferencia de otros modelos como el jerárquico y el de red).

Esto tiene la considerable ventaja de que es **más fácil de entender** y de utilizar para un usuario esporádico de la base de datos.

Características

- Una base de datos relacional se compone de **varias tablas o relaciones**.
- **No pueden** existir dos tablas con el mismo nombre.
- Cada tabla es a su vez un **conjunto de registros o filas**.
- Cada registro representa **un objeto** del mundo real.
- **No pueden existir** dos campos o columnas con el mismo nombre en la misma tabla.

- Los valores almacenados en una columna deben ser del **mismo tipo de dato**.
- Todas las filas de una misma tabla poseen el mismo **número de campos**.
- No se considera **el orden** en que se almacenan los registros en las tablas.
- No se considera **el orden** en que se almacenan las tablas en la base de datos.
- La información puede ser **recuperada o almacenada mediante "consultas"** que ofrecen una amplia flexibilidad y poder para administrar la información.

2.Tablas

Tabla en las bases de datos, se refiere al tipo de modelado de datos donde se guardan los datos recogidos por un programa. Su estructura general se asemeja a la vista general de un programa de tablas

Las tablas se componen de dos estructuras:

- **Campo o columnas:** Corresponde al nombre de la columna. Debe ser único y además tener un tipo de dato asociado.
- **Registro:** Corresponde a cada fila que compone la tabla. Allí se componen los datos y los registros. Eventualmente pueden ser nulos en su almacenamientos. Son las casillas horizontales dentro de una tabla

columna tipo numérica

id	nombre	apellido	edad	mail	celular
1	Juan	Hagan	32	juan@gmail.com	1561235689
2	Gonzalo	Fernandez	29	gonzalo@gmail.com	1598631412
3	Daniel	Allende	35	daniel@gmail.com	1598432478
4	Ana	Sacan	27	ana@gmail.com	1123987463
5	Arturo	Lopez	24	arturo@gmail.com	1147856932

fila

valor tipo texto

valor tipo numérico

3. Columnas

En el contexto de una tabla de base de datos relacional, **una columna, campo o atributo** es cada uno de los valores únicos (datos) que proporcionan la estructura según la cual se descomponen las filas (registros o tuplas) de una tabla.

Los datos de cada campo **pueden ser de diferentes tipos**, pero solo uno por columna: numéricos, alfanuméricos, lógicos (verdadero o falso), de texto, multimedia, binarios, etc.

Para evitar la inconsistencia de una estructura de campos, al diseñarla, se deben seguir las formas normales, sobre todo en bases de datos de gran tamaño.

Tabla: alumnos

id	nombre	apellido	edad	mail	celular
numérico	alfanumérico	alfanumérico	numérico	alfanumérico	alfanumérico

4. Registros

Un registro (también llamado fila) representa un objeto único de datos implícitamente estructurados en una tabla. En términos simples, una tabla de una base de datos puede imaginarse formada de filas y columnas (campos o atributos).

Cada fila de una tabla representa **un conjunto de datos relacionados**, y todas las filas de la misma tabla tienen la misma estructura. No puede haber un registro duplicado, los datos deben ser diferentes en al menos uno de los campos.

5. Tipos de datos

Una de las características principales de los campos, además de su nombre, es el tipo de dato que va a guardar. Esto define los posibles valores que se permitirán almacenar, la forma de guardarlos y hasta el espacio requerido para cada columna.

Tipos de datos con formato string

char

Son textos cuya longitud siempre será fija. Si un campo nombre es de longitud 50 y por ejemplo guardamos Flavia (6 caracteres) , los otros 44 no se ocupan.

varchar

Son textos cuya longitud es variable. Siguiendo el ejemplo anterior, si guardamos el nombre Flavia (6 caracteres) , los otros 44 no se desperdiciaron.

text

Este tipo de dato permite guardar textos muy extensos, hasta 2gb, son capaces de contener hasta un libro completo.

Tipos de datos numéricos

int (integer)

Solo permiten valores positivos o negativos sin punto decimal.

decimal (numeric)

Guarda valores con decimales.

float

número de coma flotante, este tipo de datos se suele utilizar para los valores en notación científica.

Fechas

date

Válido para almacenar una fecha con año, mes y día, su rango oscila entre '1000-01-01' y '9999-12-31'.

datetime

Almacena una fecha (año-mes-día) y una hora (horas- minutos-segundos), su rango oscila entre '1000-01-01 00:00:00' y '9999-12-31 23:59:59'.

6. Relaciones

Claves Primarias

Son campos llave que identifican unívocamente registros que están dentro de la tabla. Sólo puede existir una clave primaria por tabla y ningún campo de dicha clave puede contener valores NULL.

Claves Foráneas

Son campos llave que permiten referenciar unívocamente a un registro que está dentro de otra tabla. Es una referencia a una clave en otra tabla. Necesitan no ser claves únicas.

Ejemplo

nombre	apellido	dirección	teléfono
Pablo	Martinez	Rivadavia 2564	4567-9852
Diego	Luca	Cuba 1234	4785-2369
Alfredo	Gomez	Medrano 200	4712-9865
Silvia	Conte	Corrientes 1452	2358-8965
Adelina	Romero	Belgrano 5698	4589-5623
Pablo	Martinez	San Pedrito 111	4578-4253
Diego	Lopez	Gascón 1239	4158-7423
Jorge Darío	Gassi	Uruguay 1478	4896-3265
Nicolás	Molda	Cucha cucha 1258	4289-6547

Si alguien nos preguntara si tenemos el teléfono del cliente Pablo Martínez, porque ocurrió un problema en la entrega de un producto en su domicilio y hay que contactarse con él, le preguntaremos a la base de datos:

- ¿Cuál es el teléfono del cliente con nombre Pablo y apellido Martínez?

La respuesta sería una tabla con los siguientes datos:

nombre	apellido	dirección	teléfono
Pablo	Martinez	Rivadavia 2564	4567-9852
Pablo	Martinez	San Pedrito 111	4578-4253

La respuesta que nos devolvió la BD es totalmente válida, los dos registros cumplen con la condición que le pedimos: el cliente Pablo Martínez. Para arreglar esto, tendríamos que preguntar cuál es la dirección del Pablo Martínez del que se necesita el teléfono y volver a generar la consulta, especificando un poco más:

- ¿Cuál es el teléfono del cliente con nombre Pablo, apellido Martínez y dirección Rivadavia 2564?

Ahora la respuesta sería una tabla con los siguientes datos:

nombre	apellido	dirección	teléfono
Pablo	Martinez	Rivadavia 2564	4567-9852

Logramos conseguir sólo el dato que nos hacía falta, pero tuvimos que usar muchas condiciones para lograr que la BD nos devolviera un registro único.

Para evitar este tipo de situaciones se usan **los campos Llave o clave**: son campos (columnas) cuyo contenido va a ser único a lo largo de toda la tabla.

A fin de **evitar** que los valores que se vayan a almacenar en este tipo de campo se **dupliquen**, por lo general se usan **campos numéricos** que la base de datos maneja cuidándose siempre de asignar un número no usado anteriormente. Un claro ejemplo de esto en la vida real es el DNI, el número de la tarjeta de crédito, las cuentas bancarias, etc.

¿Cómo aplicamos esto a nuestra tabla de clientes? Lo que podemos hacer es agregar un campo llave denominado “nro cliente” que represente un Número Único de Cliente, el cual utilizaremos para referirnos unívocamente a cada uno de ellos.

Entonces la tabla quedaría así:

Tabla: PEDIDOS

nro_cliente	nombre	apellido	dirección	teléfono
1	Pablo	Martinez	Rivadavia 2564	4567-9852
2	Diego	Luca	Cuba 1234	4785-2369
3	Alfredo	Gomez	Medrano 200	4712-9865
4	Silvia	Conte	Corrientes 1452	2358-8965
5	Adelina	Romero	Belgrano 5698	4589-5623

6	Pablo	Martinez	San Pedrito 111	4578-4253
7	Diego	Lopez	Gascón 1239	4158-7423
8	Jorge Darío	Gassi	Uruguay 1478	4896-3265
9	Nicolás	Molda	Cucha cucha 1258	4289-6547

En las Bases de Datos Relacionales, como su nombre lo indica, las relaciones entre las diversas tablas que la componen tienen un rol importante. Para indicar estas relaciones de una tabla a la otra, se utilizan los campos llave.

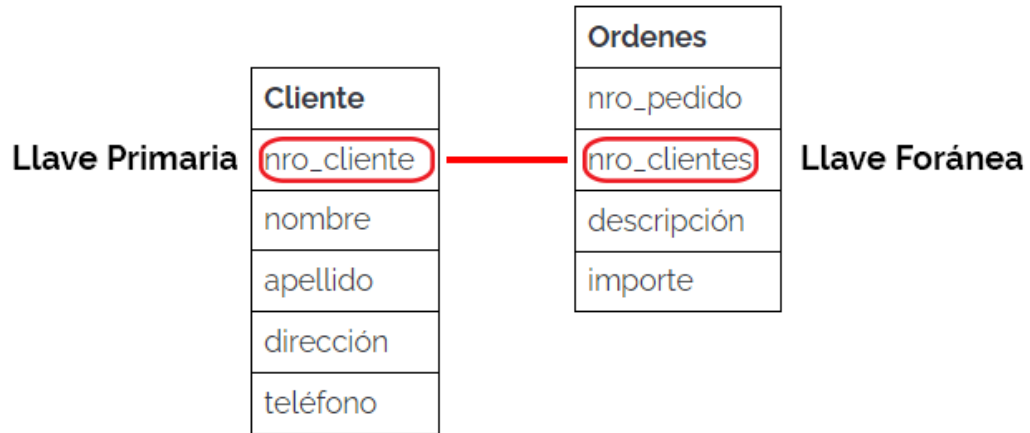
Tabla: Ordenes

nro_pedido	nro_cliente	descripción	importe
1	5	10 Resmas A4	3000
2	1	100 lapiceras	1000
3	6	22 Cuadernos Oficio	2420

Si analizamos la estructura de esta tabla, nos encontramos con dos campos llave: nro pedido y nro cliente.

Con “**nro pedido**” lo que logramos es identificar unívocamente al pedido realizado, y con “**nro cliente**”, podemos ubicar al cliente que realizó la compra.

Si queremos saber quién compró “100 Resmas A4”, sólo hace falta ver el Número de Cliente que está en la columna “**nro cliente**” y luego ir a la tabla Clientes para ver a quién le corresponde ese número, que en este caso es “Adelina Caraibo”.



Conclusión

En resumen, aprendimos que una base de datos es una colección organizada de datos estructurados, almacenados para facilitar la recuperación, gestión y manipulación de la información.

En la próxima unidad veremos...

SQL, un lenguaje de programación diseñado para gestionar y manipular bases de datos relacionales. Permite realizar diversas operaciones, como la creación, modificación, eliminación y consulta de datos.



Bibliografía utilizada y sugerida

Artículos de revista en formato electrónico:

PostgreSQL Global Development Group. (s.f.). SQL commands. PostgreSQL Documentation. <http://www.postgresql.org/docs/9.1/static/sql-commands.html>

Stabare, M. (s.f.). Tutorial de SQL. Universidad Nacional de Colombia. <http://www.unalmed.edu.co/~mstabare/Sql.pdf>